Unsupervised Learning in Genomics: Batch Effects and Single-Cell Data

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Associate Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-10-29

Learning objectives

By the end of this lecture you should be able to:

- Use PCA to spot **batch effects**, and correct them with ComBat
- Explain graph-based clustering (Louvain, Leiden) and modularity
- Run a basic single-cell workflow: PCA $\rightarrow$ nearest-neighbor graph $\rightarrow$ Leiden $\rightarrow$ UMAP
- Check clusters against known labels, and know their limits

Putting the tools to work

This lecture applies the last two lectures to genomics data:

- **Batch effects:** PCA as a diagnostic (bladder cancer microarrays)
- **Graph-based clustering:** Leiden on the tissue gene expression data
- **Single-cell RNA-seq:** the full workflow on 700 blood cells

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Batch effects

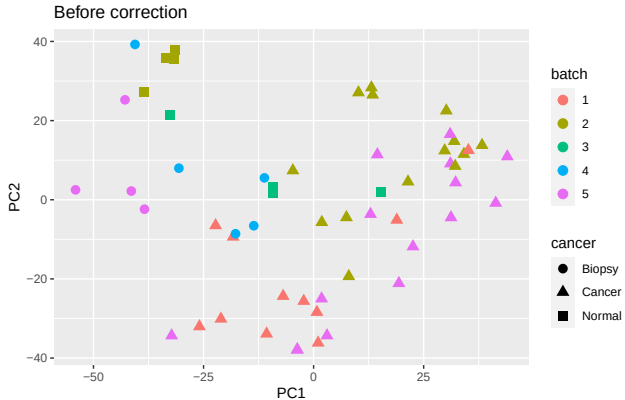
- **Batch effects:** technical variation from processing date, lab, reagent lot, or platform
- Often the **largest** source of variation in omics data
- Can hide biology, or create false findings if confounded with the outcome
- PCA is the first diagnostic: color the PC plot by batch

Batch effects: bladder cancer data

57 bladder samples (normal, cancer, biopsy) processed in 5 batches (`bladderbatch`, Bioconductor):

```
library(bladderbatch); library(sva)
data(bladderdata)
e  <- exprs(bladderEset); pd <- pData(bladderEset)
top <- order(apply(e, 1, var), decreasing = TRUE)[1:2000]
pc_plot <- function(m, main) {
  p <- prcomp(t(m[top, ]))
  data.frame(p$x[, 1:2], batch = factor(pd$batch), cancer = pd$cancer) %>%
    ggplot(aes(PC1, PC2, color = batch, shape = cancer)) +
    geom_point(size = 3) + ggtitle(main)
}
pc_plot(e, "Before correction")
```

Batch effects: bladder cancer data



Samples group by batch as much as by cancer status.

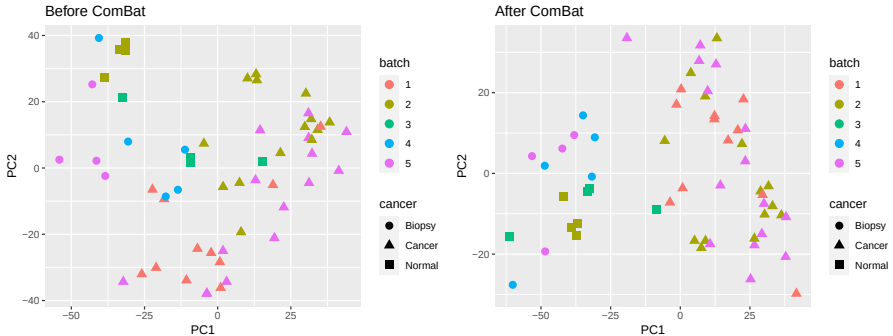
Batch correction with ComBat

ComBat (Johnson, Li & Rabinovic, 2007) removes batch-specific location and scale shifts, borrowing strength across genes (empirical Bayes). Protect the biology with `mod`:

```
mod <- model.matrix(~ cancer, data = pd)
e_cb <- ComBat(dat = e, batch = pd$batch, mod = mod)
```

For RNA-seq counts, use **ComBat-seq** (`sva::ComBat_seq`).

Batch correction with ComBat



Graph-based clustering: modularity

Standard for single-cell data (Seurat, Scanpy). Build a **nearest-neighbor graph**, then find communities that maximize **modularity**:

$$\mathcal{Q} = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

- A_{ij}: edge weight between nodes i and j; m: total edge weight
- k_i: degree of node i; $\delta(c_i, c_j) = 1$ if same community

High $\mathcal{Q}$: dense connections within communities, sparse between.

Louvain and Leiden algorithms

Louvain (greedy; repeat until $\mathcal{Q}$ stops improving):

1. **Optimize:** move each node to the neighbor community with the largest gain in $\mathcal{Q}$
2. **Aggregate:** collapse each community into a single node; repeat

Leiden (Traag et al., 2019) adds a refinement step that guarantees connected communities. Now the default in Seurat and Scanpy.

Graph-based clustering: tissue data

We return to the tissue gene expression data from the clustering lecture (189 samples, 500 genes, 7 tissues):

```
data("tissue_gene_expression")
x_tissue <- tissue_gene_expression$x
xt <- sweep(x_tissue, 2, colMeans(x_tissue))
```

Leiden clustering in R

```r
library(igraph); library(FNN)
pcs <- prcomp(xt)$x[, 1:20]                  # work in PC space
nn  <- get.knn(pcs, k = 10)$nn.index         # 10 nearest neighbors
g   <- graph_from_edgelist(cbind(rep(1:nrow(pcs), 10), as.vector(nn)),
                           directed = FALSE) %>% simplify()
set.seed(1)
cl  <- membership(cluster_leiden(g, objective_function = "modularity",
                                 n_iterations = -1))
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Leiden clustering: results

```
table(cluster = cl, tissue = tissue_gene_expression$y)
```

```
##         tissue
## cluster cerebellum colon endometrium hippocampus kidney liver placenta
##       1         32     0           0           0      0     0        0
##       2          1     0           0           0     21     0        0
##       3          5     0           0          31      0     0        0
##       4          0    34           0           0      0     0        6
##       5          0     0          15           0      0     0        0
##       6          0     0           0           0     18     0        0
##       7          0     0           0           0      0    26        0
```

```
mclust::adjustedRandIndex(cl, tissue_gene_expression$y)
```

```
## [1] 0.798626
```

Leiden clustering: lessons

- We never chose k: the **resolution** parameter controls how many communities
- Kidney splits in two, and placenta joins colon: more communities is not always better
- We use the same approach for single-cell data next

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Putting it together: single-cell RNA-seq

Standard workflow (what Seurat and Scanpy do under the hood):

1. Normalize and log-transform counts; pick highly variable genes
2. Scale genes; **PCA**; keep the top PCs
3. Build a **nearest-neighbor graph** in PC space
4. **Leiden** clustering on the graph
5. **UMAP** for visualization
6. Label clusters using marker genes

Single-cell data: 700 blood cells

PBMCs from Zheng et al. (2017), 765 highly variable genes, log-normalized (a subset distributed with Scanpy). Each cell also has a FACS-sorted cell-type label:

```
pbmc <- read.csv("../../python/data/pbmc68k_reduced.csv", check.names = FALSE)
cell_type <- pbmc$cell_type
x_sc <- as.matrix(pbmc[, -1])
dim(x_sc)
```

```
## [1] 700 765
```

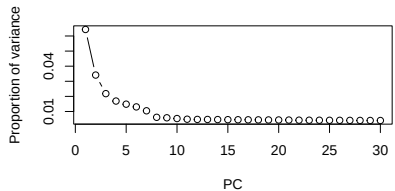
```
sort(table(cell_type), decreasing = TRUE)
```

```
## cell_type
##                     Dendritic                CD14+ Monocyte
##                           240                           129
##                      CD19+ B               CD4+/CD25 T Reg
##                            95                            68
##             CD8+ Cytotoxic T CD8+/CD45RA+ Naive Cytotoxic
##                            54                            43
##                      CD56+ NK          CD4+/CD45RO+ Memory
##                            31                            19
```

WEJ LAB
RUTGERS HEALTH Center for Data Science

Step 2: PCA

```
pc_sc <- prcomp(x_sc, scale. = TRUE, rank. = 30)
pve <- pc_sc$sdev^2 / sum(pc_sc$sdev^2)
plot(pve[1:30], type = "b", xlab = "PC", ylab = "Proportion of variance")
```



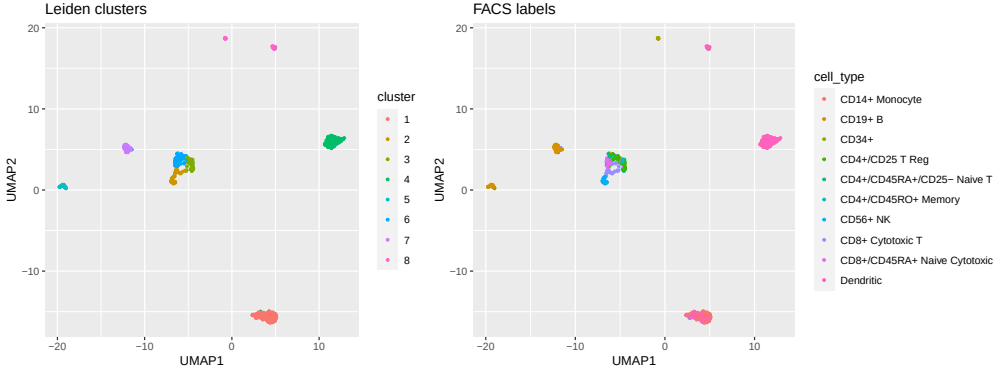
The scree plot flattens around PC 10-20; we keep 20.

Steps 3-5: graph, Leiden, UMAP

```r
library(FNN); library(igraph)
pcs_sc <- pc_sc$x[, 1:20]
nn <- get.knn(pcs_sc, k = 15)$nn.index
g  <- graph_from_edgelist(cbind(rep(1:nrow(pcs_sc), 15), as.vector(nn)),
                          directed = FALSE) %>% simplify()
set.seed(1)
cluster <- factor(membership(cluster_leiden(g, objective_function = "modularity",
                                            resolution = 1, n_iterations = -1)))
um_sc <- umap(pcs_sc, n_neighbors = 15)
table(cluster)
```

```
## cluster
##   1   2   3   4   5   6   7   8
## 178  64  65 158  33  88  66  48
```

Single-cell UMAP



Clusters vs. cell-type labels

Labels abbreviated (Mono = CD14+ monocyte, DC = dendritic, nv = naive, mem = memory):

```
table(cluster, label = short[cell_type])
```

```
##         label
## cluster   B CD34 CD4mem CD4nv CD8 CD8nv  DC Mono  NK Treg
##       1   0    0      1     0   3     0  51  123   0    0
##       2   0    0      0     0  30     4   0    0  30    0
##       3   1    0     11     2   5     6   0    0   0   40
##       4   0    0      0     0   0     0 153    5   0    0
##       5  30    0      0     2   0     0   0    1   0    0
##       6   2    0      7     3  13    33   1    0   1   28
##       7  62    0      0     1   3     0   0    0   0    0
##       8   0   13      0     0   0     0  35    0   0    0
```

```
mclust::adjustedRandIndex(cluster, cell_type)
```

```
## [1] 0.5195588
```

Single-cell: lessons

- Monocytes, B cells, and CD34+ progenitors each map to their own clusters
- T-cell subtypes overlap: their FACS markers are proteins, and their RNA profiles are similar
- NK cells and cytotoxic T cells share a cluster (shared cytotoxic genes)
- "Dendritic" labels are spread across clusters: the bulk FACS labels are noisy
- Clusters are hypotheses; marker genes and biology confirm them

WEJ LAB

Summary

- Batch effects can dominate omics data; color PCA plots by batch to find them
- ComBat removes batch shifts while protecting the biology (`mod`); ComBat-seq for counts
- Graph-based clustering (Leiden) finds communities in a nearest-neighbor graph; resolution, not k, sets the number of clusters
- Single-cell: PCA $\rightarrow$ kNN graph $\rightarrow$ Leiden $\rightarrow$ UMAP
- Clusters are hypotheses: check them against labels, marker genes, and biology

Python companion

Python versions of these examples (`combat`, `igraph`/`leidenalg`, `scanpy`):

`python/week4/genomicsApplications.ipynb`

WEJ LAB

RUTGERS HEALTH
Center for Data Science

Session Info

```
sessionInfo()
```

```
## R version 4.3.3 (2024-02-29)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
##
## locale:
## [1] C
##
## time zone: America/Denver
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] FNN_1.1.4           igraph_1.6.0        sva_3.50.0
##  [4] BiocParallel_1.36.0 genefilter_1.84.0   mgcv_1.9-1
##  [7] nlme_3.1-164        bladderbatch_1.40.0 Biobase_2.62.0
## [10] BiocGenerics_0.48.1 dslabs_0.7.6        uwot_0.1.16
## [13] Matrix_1.6-5        lubridate_1.9.3     forcats_1.0.0
## [16] stringr_1.5.1       dplyr_1.1.4         purrr_1.0.2
## [19] readr_2.1.5         tidyr_1.3.1         tibble_3.2.1
## [22] ggplot2_3.4.4       tidyverse_2.0.0
```